

Assignment 2- Case Study (8995-G level Task)

Stage 5 Tutorial Activities

Architecture and Responsibility

Week 8 | 60 minutes

Activity 1 - Where Does This Belong?

Responsibility	Layer	Reason
Read menu input	Presentation Layer	Capturing raw user input is strictly a user interface concern.
Check appointment status transition	Domain Layer	Enforcing state invariants and lifecycle rules belongs inside the domain entity.
Coordinate booking use case	Application / Controller Layer	Orchestrating multi-step workflows is the responsibility of an application controller.
Execute SQLite INSERT	Persistence / Infrastructure Layer	Performing database read/write queries and managing SQL connections is technical infrastructure layer.
Format confirmation message	Presentation Layer	Formatting data for human display belongs in the presentation layer.
Find appointment by ID	Application / Controller Layer	Querying in-memory system registries is handled by the Application/Controller Layer.

Activity 2 - Architecture Smell Hunt

A SmartCare file contains input(), SQL, appointment conflict rules, printing and validation. Identify at least five architecture problems and propose a layer for each responsibility.

- 1) Violation of the Single Responsibility Principle (SRP): Leads to fragile code where fixing a display bug could accidentally break domain rules or database execution.
- 2) Tightly Coupled Infrastructure & No Unit Testability: Cannot run automated unit tests on core business rules without spinning up a live SQLite database and simulating interactive terminal input.
- 3) Duplication & Lack of Reusability: Business logic trapped alongside input() and print() statements cannot be reused if the system expands
- 4) Bypassed Encapsulation & Domain Invariants: When persistence and input validation happen in the same procedural script as conflict rules, state transitions and invariant rules are easily bypassed or inconsistently enforced across different parts of the script.
- 5) Leaky Abstractions & Security Risks: Mixing user input parsing directly with database access increases the risk of SQL injection or unhandled database exceptions crashing the user interface.

Activity 3 - SOLID Without Overengineering

ClinicManager handles every use case. Which principle is threatened?

Single Responsibility Principle (SRP)

AppointmentService imports sqlite3 directly. What dependency concern exists?

Violation of the Dependency Inversion Principle

A repository interface has 20 methods but a client needs two. What concern exists?

Violation of the Interface Segregation Principle

Should every class have an interface? Explain.

No, symptom of overengineering

Activity 4 - AI Architecture Critique

AI proposes microservices, an event bus, six interfaces and a dependency-injection framework. Decide what to reject, defer or keep using current requirements.

Proposal	Decision	Rationale
Dependency Injection (DI) Framework	Reject	Pulling in a framework adds unnecessary library dependencies, configuration overhead, and double underscore methods.
Microservices Architecture	Reject	SmartCare is currently a single domain model operating in a single process. Microservices introduce network latency, distributed transactions, independent deployment pipelines, and complex service discovery.
Event Bus / Async Messaging	Defer	Asynchronous event-driven messaging is useful for decoupling side effects like sending SMS reminders or billing notifications. However, since current requirements focus on core synchronous CRUD workflows, state transitions, and collision checking, an event bus adds premature complexity.
Six Separate Interfaces	Defer	Why Defer Most: Creating interfaces for core domain entities is overengineering or "interfaceitis."